

Kod Tabanı Soru-Cevap Asistanı

Staj ara raporu · Yusuf Güneş · 25.08.2026

Bir kod tabanını indeksleyip doğal dildeki sorulara **dosya ve satır referanslı** cevap veren bir araç geliştirildi. Amaç, yeni bir müşteri projesine adapte olma süresini kısaltmak: her cevap kaynak gösterdiği için doğruluğu anında kontrol edilebiliyor.

Ölçülen sonuç: 1097 dosyalık üçüncü taraf bir kod tabanında, ayarların hiç dokunulmadığı 20 soruluk bir test setinde **cevap doğruluğu %100**, uydurulmuş kaynak referansı yok. Araç sekiz programlama dilini okuyabiliyor ve Claude Code'un içinden kullanılabiliyor.

1. Neye ulaşıldı

Hafta	Hedef	Durum
1	Kod sembolü çıkarma, doküman parçalama	Tamamlandı
2	Embedding ve arama katmanı	Tamamlandı
3	Cevaplama katmanı, ölçüm düzeneği, iyileştirmeler	Tamamlandı
4	Claude Code / MCP entegrasyonu, çok dil desteği	Tamamlandı
—	Demo ve sunum	Sırada

Sayılarla

Ölçüt	Değer
Desteklenen dil	8 (Python, C, C++, Java, C#, Go, TypeScript, JavaScript)
Birim testi	182 — tamamı ağ erişimi ve API anahtarı olmadan çalışıyor
Ölçüm soru seti	100 soru (20 kendi repo + 60 kolay + 20 akış)
Toplam API harcaması	yaklaşık 10 ABD doları

2. Sistem nasıl çalışıyor

Kod, sabit uzunlukta metin blokları yerine **sözdizimi ağacı üzerinden** bölünüyor. Her fonksiyon, metot ve sınıf ayrı bir parça oluyor. Bunun iki sebebi var: parçalar anlamlı bir bütün olarak kalıyor ve her parçanın gerçek satır aralığı biliniyor — kaynak referansı verebilmenin ön şartı bu.

Soru geldiğinde önce arama katmanı çalışıyor, bulunan parçalar bağlam olarak Claude'a veriliyor. Model eksik kalan yerleri iki araçla kendisi okuyor: dosya okuma ve ikinci tur arama. Cevaptaki her referans, dosyanın var olduğuna ve satırın dosya sınırları içinde kaldığına göre programla doğrulanıyor.

Güvenlik notu: modelin okuyacağı dosya yolu güvenilmez girdi kabul ediliyor ve repo kökünün dışına çıkma denemeleri reddediliyor. Bu davranış testlerle sabitlendi.

3. Ölçüm düzeneği

Ölçüm, aracın yan ürünü değil parçası. Sebebi şu: **doğru yapılandırma kod tabanına göre değişiyor** ve bunu ancak ölçerek bulmak mümkün. Düzenek iki şeyi ayrı ölçüyor:

Ne ölçülüyor	Nasıl	Maliyet
Getirme — arama doğru parçayı ilk sonuçlara soktu mu	Beklenen dosya elle işaretlenmiş sorular	Ücretsiz
Cevap — model doğru dosyaya referans verdi mi	Aynı sorular, uçtan uca cevap üretilerek	Ücretli

Getirme bozuksa cevap da bozulur, ama tersi doğru değil: arama doğru parçayı getirip model yine de yanlış cevap verebilir. Ayrı ölçülmezse hangisinin düzeltileceği belli olmuyor.

Ayrıca **dev/test ayrımı** var: ayar denemeleri yalnızca dev bölmesinde yapılıyor, rapor edilen sayı hiç dokunulmamış test bölmesinden alınıyor. Aynı sorular üzerinde hem ayar yapıp hem rapor etmek, ayarı o setin gürtlüsüne uydurmak demek.

4. Sonuçlar

4.1 Cevap doğruluğu (test bölmesi, 20 soru)

Ölçüt	Sonuç
Doğru cevap oranı	%100
Beklenen dosya kapsamı	%100
Uydurulmuş referans	0

Dikkat çeken bulgu: bir soruda arama doğru parçayı ilk sekize sokamadı, ama cevap yine doğru çıktı — model kendi arama araçlarıyla eksiği kapattı. Yani getirme başarımı tek başına kalite göstergesi değil; araç katmanının varlık sebebi tam olarak bu.

4.2 Getirme başarımının gelişimi

Aşama	Bulma	Sıralama (MRR)
Başlangıç	%87	0,714
Modül seviyesi sabitler indekslendi	%88	0,727
Soru etiketleri düzeltildi *	%93	0,761

* Bu satır bir sistem iyileştirmesi **değil**, ölçüm düzeltmesi: cevabı birden fazla dosyada olan sorulara tek dosya yazılmıştı, araç doğru cevabı getirdiği hâlde hata sayılıyordu. Kazanç diye sunulması yanıltıcı olur.

4.3 Akış soruları — asıl vaadin sınanması

İlk soru setinin tamamı “şu şey nerede” tipindeydi ve cevap doğruluğu tavana vurdu; bu noktadan sonra hiçbir iyileştirmenin işe yarayıp yaramadığı ölçülemez hâle geldi. Oysa projenin asıl vaadi akış takibi: bir işin nerede başlayıp nereden geçtiğini anlatmak. Bunun için cevabı 2–4 dosyaya yayılan 20 soruluk ikinci bir set hazırlandı.

Soru seti	Bulma	Sıralama	Kapsam
Kolay (tek konum, 60 soru)	%93	0,761	%92

Zor (akış, 20 soru)	%100	0,846	%80
---------------------	------	-------	-----

Zor sette “bulma” oranının daha yüksek çıkması iyi haber değil — **metriğin yanlış olduğunu gösteriyor**. Çok dosyalı bir soruda “en az bir doğru dosya bulundu” demek kolay: üç dosyadan birini bulmak yetiyor. Asıl ölçüt kapsam ve orada %92’den %80’e düşüyor. Yani araç akış sorularında da doğru yere gidiyor, fakat zincirin tamamını getiremiyor: ortalama her beş dosyadan biri sonuçların dışında kalıyor. Bundan sonraki iyileştirme çalışmasının ölçüleceği yer burası.

4.4 Embedding sağlayıcısı karşılaştırması

Model	Nerede çalışıyor	Bulma	Sıralama
voyage-code-3	Bulut	%100	0,702
bge-m3	Yerel (Ollama)	%100	0,654
nomic-embed-text	Yerel (Ollama)	%65	0,302

5. Öğrenilenler

Bu bölüm sonuçların en değerli kısmı: neyin **işe yaramadığı** da veriyle biliniyor. Aşağıdaki maddelerin her biri denenmiş, ölçülmüş ve karar veriyeye dayandırılmıştır.

Hibrit arama bedava bir kazanç değil

Tasarım varsayımı, anahtar kelime araması ile anlamsal aramanın birbirini tamamlayacağıydı. Küçük ve Türkçe yorumlu bir kod tabanında bu doğrulandı. Ancak 4311 parçalık İngilizce bir kod tabanında **saf anlamsal arama hibriti geçti**: anahtar kelime aramasının başarımı %95’ten %15’e düştü. Sebep dil farkı — Türkçe soru kelimeleri İngilizce kodda hiç geçmiyor, dolayısıyla kelime eşleşmesinin tutunacağı yer kalmıyor. **Çıkarım**: arama modu sabitlenmemeli, her müşteri kod tabanında ölçülüp seçilmeli.

Yerel model kullanılabilir — ama model seçimi belirleyici

İlk denemede yerel modelle başarıım %65’te kaldı ve “yerel model kalite kaybettiriyor” sonucuna varmak mümkündü. Farklı bir yerel modelle aynı ölçüm %100 verdi. Kaybettiren yerel olmak değil, zayıf modeldi. Bu, **müşteri kodunun dışarı çıkamadığı projelerde aracın kullanılabilir olduğu** anlamına geliyor.

Contextual retrieval masrafını çıkarmadı

Her parça için bir dil modeline bağlam cümlesi yazdırma tekniği uygulandı ve ölçüldü: anlamsal aramada +2 puan, maliyet 5–7 dolar ve saatlerce işlem. Anahtar kelime tarafını belirgin şekilde iyileştirdi (%15 → %47) ama asıl kullanılan yol anlamsal arama. Muhtemel sebep, karşılaştırma tabanının zaten zayıf olmaması: parçalar sözdizimi ağacıyla bölündüğü için bütünlüğünü koruyor ve hâlihazırda bir konum etiketi taşıyor. Tekniğin asıl değeri, sabit uzunlukta bölünmüş ve bağımsız parçalarda ortaya çıkıyor.

Denenip kapatılan: üretilmiş tip tanımlarını geri plana atmak

İncelenen kod tabanının %42’si otomatik üretilmiş tip tanımı ve bunlar bazı sorularda gerçek cevabı sonuçların dışına itiyor. Bu parçaların ağırlığını düşürmek mantıklı görünüyordu, ancak ölçümde sıralama başarımını düşürdü. Kod korundu, varsayılan olarak kapatıldı. Ayrıca “otomatik üretilmiş dosya” işareti ayırt edici bir sinyal değil: bu kod tabanındaki dosyaların %92’si o işareti taşıyor, sorulara cevap veren dosyalar dahil.

Araç kendi kod tabanındaki bir hatayı buldu

Geliştirme sırasında araca kendi koduyla ilgili bir soru soruldu ve cevabında gerçek bir hataya işaret etti: önbellek anahtarı, embedding'e giden metnin tamamından değil yalnızca bir bölümünden hesaplanıyordu. Bunun sonucu, bir dosya taşındığında önbellekten eski bağlamla üretilmiş vektörün dönmesiydi. Hata doğrulandı ve düzeltildi; etkisi ölçülebilir oldu (4310 parça daha önce 3549 vektöre çöküyordu).

6. Sıradaki adımlar

Öncelik	İş	Maliyet
1	Akış sorularında kapsamı %80'in üzerine çıkarmak	Düşük
2	Python dışındaki dillerde doğruluk ölçümü	Orta
3	Demo ve sunum hazırlığı	Yok
4	Akış setinde uçtan uca cevap ölçümü	Orta

Birinci madde için planlanan yöntem: soruyu doğrudan aramak yerine, soruya cevap olabilecek örnek bir kod parçası ürettirip onu aratmak. Ölçülen dil farkı sorununa doğrudan saldırıyor, çünkü üretilen metin kodun kendi biçiminde oluyor.

7. Açık konular

Aşağıdaki iki konu teknik değil, yönlendirme gerektiriyor:

- **Müşteri projeleri hangi dillerde yazılıyor?** Araç sekiz dili okuyabiliyor, ancak doğruluk yalnızca Python üzerinde ölçüldü. Ağırlıklı dil bilinirse ölçüm oradan tekrarlanır.
- **Yerel model bir zorunluluk mu, yoksa karşılaştırma seçeneği mi?** Mimari her iki cevaba da hazır ve yerel seçeneğin kalite maliyeti ölçüldü. Cevap, varsayılan yapılandırmayı belirler.

Rapordaki bütün sayılar, kod deposundaki ölçüm düzeneğiyle üretilmiş ve koşu çıktıları diske kaydedilmiştir; tekrar üretilebilir durumdadır.